

Ex-no: 5	SYSTEM CALLS
12/06/2026	

Aim

To study and implement basic system calls in Linux for performing operations like file handling and process management.

Algorithm

1. Open the Linux terminal.
2. Write a C program that uses system calls (such as `open()`, `read()`, `write()`, `close()`).
3. Save the program in a file (e.g., `syscall.c`).
4. Compile the program using `gcc syscall.c -o syscall`.
5. Execute the compiled program using `./syscall`.
6. Observe the output and verify the working of system calls.

CODE:

Pid.c:

```
#include <stdio.h>
#include <unistd.h>

int main() {

    printf("PID: %d\n", getpid()); // PID of the current process
    printf("PPID: %d\n", getppid()); // PID of the process that started
    your program

    return 0;
}
```

Readdir.c:

```
#include <stdio.h>
#include <dirent.h>

int main() {

    DIR *dir;
    struct dirent *entry;

    dir = opendir(".");

    if (dir == NULL) {
```

```

        printf("Cannot open directory\n");
        return 1;
    }

    printf("Files in directory:\n");

    // Loop through all files in the directory
    while ((entry = readdir(dir)) != NULL) { // readdir() reads one entry
at a time
        printf("%s\n", entry->d_name); // d_name stores the file name
    }

    closedir(dir);

    return 0;
}

```

Opendir.c:

```

#include <stdio.h>
#include <dirent.h> // for DIR, opendir, closedir

int main() {

    DIR *dir;

    dir = opendir("."); // opendir() opens a directory. "." means the
current directory

    if (dir == NULL) { // if opendir() fails, it returns NULL
        printf("Failed to open directory\n");
        return 1; // exit program with error code
    }

    printf("Directory opened successfully!\n"); // directory opened

    closedir(dir); // close the directory

    return 0;
}

```

Fork.c:

```

#include <stdio.h>
#include <unistd.h>

int main() {

    // child process execute the if block
    if(fork() == 0) {
        printf("This is CHILD process\n");
    }

    // parent process execute the else block
    else {

```

```
        printf("This is PARENT process\n");
    }

    printf("PID: %d\n", getpid());

    return 0;
}
```

Exec.c:

```
#include <stdio.h>
#include <unistd.h>

int main() {

    printf("Before exec()\n");

    execl("/bin/ls", "ls", NULL);

    printf("After exec()\n");

    return 0;
}
```

OUTPUT:

./a.out

PID: 2456

PPID: 2380

Directory opened successfully!

Result

The system calls were successfully implemented using a C program. The program executed correctly, demonstrating file operations and interaction with the operating system through system calls.